

SAI TEJA EPURI

+91 6281615355 | saitejae035@gmail.com

PROFILE

Enthusiastic and DevOps aspirant with strong knowledge of version control (Git), containerization using Docker, and orchestration with Kubernetes. Skilled in setting up CI/CD pipelines using Azure DevOps and Azure Pipelines to streamline software delivery. Familiar with automating infrastructure management on Azure. Passionate about building scalable, reliable, and efficient cloud solutions, with a commitment to continuous learning and hands-on practice.

EDUCATION

Degree (B.Tech-CSE)	2021 - 2025
----------------------------	-------------

SAVEETHA SCHOOL OF ENGINEERING
(CGPA - 8.4)

Intermediate/Diploma	2019 - 2021
-----------------------------	-------------

NARAYANA JUNIOR COLLAGE
(Percentage - 87.4)

SSC, General Studies	2009 - 2019
-----------------------------	-------------

SRI CHAITANYA TECHNO SCHOOL
(CGPA - 9.2)

SKILLS

VCS: Git & GitHub

OS: Linux

Containerization: Docker

Orchestration: Kubernetes

Cloud Computing: Azure Cloud & Azure DevOps

Build Tools/Package Manager: Maven, NPM, Nuget, MsBuild

Web Server: Tomcat, Nginx

CI/CD: Azure Pipelines

IaC: Terraform

WORKING KNOWLEDGE:

- Efficiently manage code versions and track changes across multiple teams.
- Streamlined branching and merging strategies to ensure clean code integration.
- Collaborate with developers by utilizing pull requests and code reviews.
- Integrate with CI/CD pipelines for automated testing and deployments.
- Use GitHub Actions to automate workflows like build, test, and deployment tasks.

- Managed code versions effectively using Git, ensuring smooth collaboration by handling branching, merging, and conflict resolution across development teams.
- Created and managed feature branches, performed merges, and resolved conflicts to integrate changes from different developers while ensuring the stability of the main codebase.
- Collaborated within GitHub workflows, actively participating in pull requests, performing code reviews, and following best practices for version control to maintain code quality.
- Administer and troubleshoot Linux-based servers for application deployments.
- Utilize command-line tools to manage files, processes, and permissions.
- Create and manage Docker images to encapsulate applications and their dependencies.
- Build and run containers for consistent environments across development, testing, and production.
- Use Docker Compose to manage multi-container applications.
- Automate the containerization process as part of CI/CD pipelines for quicker deployments.
- Optimize Dockerfiles for efficient builds and minimized image sizes.
- Manage containerized applications at scale with Kubernetes clusters.
- Implement Kubernetes services, deployments, and pods for orchestration.
- Monitor and troubleshoot Kubernetes clusters to ensure high availability.
- Automate scaling and self-healing of applications using Kubernetes features.
- Perform system monitoring, performance tuning, and logging with native tools.
- Deploy applications and manage configurations on Linux servers.
- Secure Linux systems through firewalls, users, and permission management.
- Maintained a clean commit history by writing clear commit messages, using git rebase and git cherry-pick to manage code changes, and utilizing git revert to undo changes when necessary.
- Implemented Git hooks to automate pre-commit checks, such as linting, testing, and formatting, to ensure code quality before committing changes to the repository.
- Developed and maintained Docker containers for consistent development, testing, and production environments, ensuring seamless deployment across different stages.
- Designed and optimized Dockerfiles to build lightweight, efficient container images for various applications, streamlining the deployment process.
- Utilized Docker Compose to manage multi-container applications, simplifying orchestration and ensuring dependencies like databases and services were properly linked.
- Automated deployment workflows by integrating Docker with CI/CD pipelines, enabling faster and more reliable application deployment.
- Managed containerized applications in production environments, monitoring container health, scaling, and troubleshooting performance issues to ensure high availability and reliability.
- Gained hands-on experience in deploying and managing applications on Kubernetes clusters, using basic Kubernetes concepts such as Pods, Deployments, and Services.
- Assisted in setting up and configuring Kubernetes clusters, working with tools like kubectl to troubleshoot and manage deployments in development and staging environments.
- Supported the scaling of applications in Kubernetes, ensuring efficient resource usage by adjusting replica counts and configuring Horizontal Pod Autoscalers (HPA).
- Contributed to automating the deployment pipeline by integrating Kubernetes with CI/CD tools, enabling continuous delivery of containerized applications.
- Gained experience in monitoring and troubleshooting Kubernetes environments, using tools like kubectl logs and monitoring solutions.
- Assisted in setting up and configuring Azure Pipelines for automating the build and

release processes, enabling continuous integration and continuous delivery (CI/CD) for applications

- Created and maintained YAML-based pipeline definitions, ensuring seamless deployment workflows for various environments such as development, staging, and production.
- Integrated Azure Pipelines with version control systems like GitHub and Azure Repos to automate code deployments and trigger builds based on code changes.
- Assisted in configuring pipeline triggers, approval gates, and manual intervention steps, ensuring smooth and controlled releases to production environments.
- Monitored pipeline performance and logs to identify issues, troubleshoot failures, and optimize build times, improving the overall efficiency of the CI/CD process.
- Assisted in automating configuration management tasks using Ansible playbooks, ensuring consistent and repeatable server configurations across development and production environments.
- Created and maintained Ansible roles and tasks to automate software deployment, system updates, and application configuration, improving operational efficiency.
- Managed infrastructure as code (IaC) by writing and executing Ansible playbooks for provisioning and managing servers in cloud environments like AWS and Azure.
- Integrated Ansible with CI/CD pipelines to automate server provisioning and configuration during the deployment process, reducing manual intervention.
- Gained hands-on experience in using Ansible for managing and orchestrating multiple systems, troubleshooting playbook execution, and optimizing playbooks for better performance and scalability.
- Managed Kubernetes namespaces to logically separate environments (e.g., dev, staging, production) and configured resource quotas to ensure fair allocation of cluster resources across teams.
- Gained experience in using taints and tolerations to control pod scheduling, ensuring that certain workloads are assigned to specific nodes and maintaining efficient resource utilization across the cluster.
- Worked with Persistent Volumes (PVs) and Persistent Volume Claims (PVCs) to manage stateful applications, ensuring data durability and scalability across pods.
- Assisted in configuring Kubernetes network policies to control traffic flow between pods, enhancing the security of the cluster by limiting communication between services.
- Configured and managed Ingress Controllers for routing external traffic to services within the cluster, ensuring secure and efficient management of incoming HTTP/HTTPS requests.
- Assisted in setting up and configuring Azure Pipelines for automating build, test, and release workflows, ensuring faster and more efficient code deployments.
- Gained hands-on experience creating and managing YAML-based pipeline definitions to automate the build and release process for multiple environments (dev, staging, production).
- Integrated Azure Pipelines with version control systems like Azure Repos and GitHub, automating code deployments and ensuring continuous integration with source code changes.
- Managed build and release pipelines, automating software builds, running unit tests, and deploying applications to Azure environments, reducing manual intervention.
- Implemented approval gates and manual intervention steps within Azure Pipelines, ensuring a controlled and secure deployment process before pushing changes to production.
- Created multi-stage Azure Pipelines to automate end-to-end workflows, from building and testing code to deploying it across different environments like dev, test, and production.

- Understand the two main types – Public Load Balancer (exposes services to the internet) and Internal Load Balancer (used for internal traffic within a virtual network).
- Configure frontend IPs for load balancing and ensure the correct public or private IP is assigned depending on the use case.
- Use health probes to monitor the health of backend services. Configure HTTP, TCP, or custom probes to ensure only healthy instances handle traffic.
- Define backend pools that consist of virtual machines, virtual machine scale sets, or availability sets. Properly managing these pools ensures high availability and scalability.
- Configure load balancing rules to define how incoming traffic is distributed across the backend pool. This includes setting up the protocol, port, and distribution method (like round-robin or least connections).
- Types of Storage Accounts: Understand the different storage account types—General-purpose v2, BlobStorage, FileStorage, and Premium—and choose the right one based on performance and use case requirements.
- Blob Storage: Familiarize with Azure Blob Storage, which is designed for unstructured data like documents, images, and backups. Know the different blob types: Block Blobs, Append Blobs, and Page Blobs.
- Azure Files: Learn to use Azure File Storage for fully managed file shares accessible via SMB and NFS protocols, making it suitable for migrating on-premise file shares to the cloud.
- Data Redundancy Options: Understand replication options like Locally Redundant Storage (LRS), Geo-Redundant Storage (GRS), Zone-Redundant Storage (ZRS), and Read-Access Geo-Redundant Storage (RA-GRS) for ensuring data availability and durability.
- Set up and configure various Ingress Controllers (NGINX, Traefik, HAProxy) in Kubernetes clusters, including service accounts and roles.
- Implement advanced routing rules with path-based, host-based, and subdomain routing, and use regex-based routing for traffic management.
- Manage TLS termination at the Ingress controller, using Kubernetes secrets or tools like Cert-Manager for automatic certificate provisioning and renewal.
- Scale Ingress Controllers horizontally to handle varying traffic patterns, utilizing horizontal pod autoscaling (HPA) for automatic adjustment.
- Secure Ingress resources by enforcing RBAC, IP whitelisting, rate limiting, and implementing authentication mechanisms such as OAuth or basic authentication.
- Acquired working knowledge of deploying infrastructure using Terraform on Azure/AWS.
- Hands-on with Terraform workflow: init, plan, and apply for provisioning resources.
- Basic understanding of Terraform state file management for tracking infrastructure changes.
- Configured remote backend on AWS S3 with DynamoDB for state locking in practice projects.
- Used variables, .tfvars files, and TF_VAR environment variables for configuration flexibility.
- Practiced passing variables via -var flag during deployments.
- Created Terraform modules for reusable and organized infrastructure code

(

PROJECTS:

- 1. Frontend Deployment Build Using Maven with CI/CD On Azure (06/2025)**
 - Developed and deployed a Frontend Application Using Maven as the Build Automation.
 - Designed and Implemented a CI/CD Pipeline in Azure DevOps to automate Build, Test, and Deployment Stages.
 - Integrated GitHub with Azure Pipelines to trigger builds on Code Commits.
 - Deployed the Application on Azure Kubernetes Services (AKS) Using Containerized Workloads.

- 2. Deployment of Three-Tier Application on AKS Using Pipelines (08/2025)**
 - Built a Complete Three-tier Architecture (Frontend, Backend, Database) and Implemented full CI/CD Pipelines Using Azure DevOps.
 - Containerized each Tier Using Docker, Stored Images in ACR, and orchestrated deployments on Azure Kubernetes Services (AKS).
 - Automated Environment Provisioning Using YAML Pipelines and Kubernetes manifests, (Deployments, Services, ConfigMaps, Secrets).
 - Integrated Monitoring and logging Using Azure Monitor.

- 3. Deployment of Three-Tier Application Using Python (Flask) (09/2025)**
 - Developed a Three-Tier Web Application with React.js as Frontend and Flask (Python) as Backend with database integration managed in backend services.
 - Containerized the complete application using Docker to ensure portability, consistency, and efficient deployment across environments.
 - Designed and structured backend services to handle database connectivity, request handling, and API communication securely and efficiently.
 - Integrated Monitoring and logging Using Azure Monitor.

- 4. Deployment of Three-Tier Application Using GitHub CI/CD (11/2025)**
 - Implemented GitHub Actions-based CI/CD pipelines to automate build and deployment workflows for both frontend and backend.
 - Configured pipelines to trigger on every push, performing NuGet restore, MSBuild compilation, and validation steps.
 - Deployed Next.js frontend and .NET backend APIs to Azure App Service, ensuring reliable and scalable hosting.
 - Used secure publish profiles, secrets management, and environment configurations to enable safe, production-ready deployments.